

Randomized Selection (Quickselect) and Week 14 Consolidation

Programming and Data Structures — Week 14, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement quickselect and use it to find the k -th smallest element and the median of an unsorted list in expected linear time; explain precisely why quickselect's expected running time is $O(n)$ even though its structure closely resembles quicksort, which runs in $O(n \log n)$; compare quickselect against the simpler alternative of sorting the entire list and then indexing into it, as two solutions to the same selection problem; and consolidate Week 14's three preceding lectures — parallel thinking and Amdahl's Law, the Las Vegas/Monte Carlo correctness-versus-speed distinction, and parallel algorithm design patterns — into a single coherent picture of this course's final algorithmic module.

Outline

This lecture proceeds through the selection problem itself — finding the k -th smallest element of a list without fully sorting it; quickselect, derived directly from quicksort's own partition step; an explanation of why quickselect achieves expected $O(n)$ running time rather than quicksort's $O(n \log n)$; a worked trace verified against real running code; a comparison of quickselect against simply sorting the list and indexing into it; an MTech addition on the guaranteed-worst-case-linear-time median-of-medians algorithm; a consolidation of Week 14's three lectures into one coherent picture; and closes with an exercise, quiz, and summary.

The problem: the k -th smallest element

Given an unsorted list of n numbers, the selection problem asks for the k -th smallest element — for instance, the median corresponds to $k = n/2$. The obvious approach sorts the entire list, which costs $O(n \log n)$ using merge sort or quicksort from Weeks 5 and 6, and then simply indexes into position k of the sorted result. This lecture answers the natural follow-up question: since only a single position is

actually needed, not the full sorted ordering of every element, can this be done faster than fully sorting first?

Quickselect, derived from quicksort's partition step

```
import random

def quickselect(arr, k):
    if len(arr) == 1:
        return arr[0]
    pivot = random.choice(arr)
    lows = [x for x in arr if x < pivot]
    highs = [x for x in arr if x > pivot]
    pivots = [x for x in arr if x == pivot]
    if k < len(lows):
        return quickselect(lows, k)
    elif k < len(lows) + len(pivots):
        return pivot
    else:
        return quickselect(highs, k - len(lows) - len(pivots))
```

Quicksort, covered in Week 6, partitions the array around a randomly chosen pivot and then recurses on both the low side and the high side of that partition. Quickselect partitions in exactly the same way, but recurses on only the one side known to contain the target position k — the other side is discarded entirely, since it is already known not to contain the answer. This single change, discarding one side instead of recursing into both, is precisely what turns an $O(n \log n)$ algorithm into one with expected $O(n)$ running time.

Why quickselect is expected $O(n)$, not $O(n \log n)$

Quicksort recurses on both halves of every partition at every level of recursion, and since there are $\log n$ levels each performing $O(n)$ total partitioning work across that level, the total cost is $O(n \log n)$. Quickselect recurses on only one half, and with randomly chosen pivots, that surviving half is expected to shrink by a roughly constant fraction at each step. The total work across all recursive calls therefore forms a geometric series, $n + n/2 + n/4 + \dots$, which converges to $O(n)$ in expectation, rather than being multiplied by $\log n$ separate levels the way quicksort's work is. This is the precise mathematical reason quickselect achieves linear expected time despite looking, at first glance, almost identical to quicksort.

Worked trace, verified against real code

```
arr = [7, 10, 4, 3, 20, 15]
sorted_arr = sorted(arr)
for k in range(len(arr)):
    assert quickselect(arr, k) == sorted_arr[k]
print("quickselect matches sorted array at every position:", sorted_arr)
```

For the array $[7, 10, 4, 3, 20, 15]$, whose sorted order is $[3, 4, 7, 10, 15, 20]$, requesting position $k = 3$ (using 0-indexing, so this is the 4th smallest element) should return 10. Running the quickselect implementation above and checking it against every position from $k = 0$ through $k = 5$ confirms it matches the fully sorted array's value at every single position exactly, verifying the algorithm's correctness directly through code execution rather than by inspection alone.

Quickselect versus sort-then-index

Sorting the entire list and then indexing into position k costs $O(n \log n)$ but yields the full sorted order of every element as a byproduct. Quickselect costs only expected $O(n)$ but yields information about just the one requested position, discarding the relative order of most other elements along the way. The correct choice depends on the actual requirement: if only a single position is ever needed, quickselect is asymptotically faster and should be preferred; if the full sorted order will be needed anyway, for instance to answer several different selection queries later, sorting the list once upfront is the better engineering choice.

MTech addition — guaranteed-worst-case-linear-time selection

Quickselect's $O(n)$ running time bound holds only in expectation — an adversarial or simply unlucky sequence of chosen pivots can still produce $O(n^2)$ worst-case behavior, exactly as with ordinary quicksort. The median-of-medians algorithm addresses this by choosing pivots more carefully: it recursively estimates a provably good pivot by grouping the elements into groups of five, finding each group's median, and recursing on those group medians, which guarantees worst-case $O(n)$ running time unconditionally, without relying on randomness at all. The practical trade-off is that median-of-medians carries substantially larger constant factors in practice, so ordinary quickselect using random pivots is usually faster on real workloads despite its comparatively weaker worst-case guarantee — a genuine engineering trade-off between guaranteed and expected performance, echoing the Las Vegas discussion from Lecture 2.

Week 14 consolidation: one coherent picture

Lecture 1 introduced parallel thinking and Amdahl's Law, establishing that some problems parallelize naturally while others fundamentally cannot, and that any sequential remainder in a computation always bounds the real, achievable speedup regardless of how many workers are available. Lecture 2 introduced randomization as a tool for navigating correctness-versus-speed trade-offs, distinguishing Las Vegas algorithms, which are always correct but have variable running time, from Monte Carlo algorithms, which are fast but carry a small, controllable error probability. Lecture 3 introduced concrete parallel algorithm design patterns: the divide-and-conquer-to-parallel translation, and the reduction pattern underlying MapReduce-style systems. This lecture, the fourth, introduced randomization used for a different purpose than Lecture 2's — not managing a correctness trade-off, but discarding an entire half of the remaining work at each step, based purely on a random pivot choice, in order to save time. Together, these four lectures show that parallelism and randomization are two distinct but complementary tools for making algorithms faster than their naive, fully sequential and fully deterministic counterparts.

A second worked trace: finding the median

```
arr2 = [9, 1, 8, 2, 7, 3, 6]
median_index = len(arr2) // 2
result = quickselect(arr2, median_index)
assert result == sorted(arr2)[median_index] == 6
print("Median found via quickselect:", result)
```

For the seven-element array $[9, 1, 8, 2, 7, 3, 6]$, whose sorted order is $[1, 2, 3, 6, 7, 8, 9]$, the median sits at index $k = 7 // 2 = 3$ of the sorted array, which holds the value 6. Running `quickselect` with $k = 3$ on the original unsorted array and checking the result directly against the sorted array's middle position confirms it correctly returns 6, verifying the algorithm finds the median without ever fully sorting the array.

Where this matters in practice

NumPy's `numpy.partition` and `numpy.median` functions use selection algorithms internally rather than performing a full sort, specifically for the efficiency reason developed in this lecture. Database query engines similarly use selection algorithms to answer “find the top- k rows” queries without needing to sort an entire table first. More generally, in any situation where only a single rank statistic is needed — a median, a 90th percentile cutoff, or the top- k elements — selection algorithms beat full sorting asymptotically, exactly as `quickselect` beats `sort-then-index` in this lecture's analysis.

Space and in-place considerations

The `quickselect` implementation shown in this lecture builds new lists for the low, high, and pivot-equal groups at every recursive call, which is simple to read and reason about but uses $O(n)$ extra space at each level of recursion. A production-quality implementation instead partitions the array in place, swapping elements by index within the original array, exactly as Week 6's in-place quicksort partition does. This distinction affects only space efficiency; the asymptotic time analysis developed earlier in this lecture is identical either way, since it depends on how much data survives to the next recursive call, not on whether that data is copied into a new list or partitioned in place.

Second quiz — sort-then-index or quickselect?

Consider a web application that needs to repeatedly answer “what is the current median response time?” as new latency measurements arrive continuously throughout the day, with the median queried far more often than any single measurement is inserted. Should the application run `quickselect` fresh on the full dataset for every query, or maintain an incrementally sorted (or partially sorted) structure instead? Justify the choice using this lecture's comparison of `quickselect` against sort-then-index.

Second quiz — answer

Neither `quickselect` nor a full one-time sort is ideal here: since the median is queried repeatedly against continuously changing data, the best solution is usually a structure that maintains order incrementally as new measurements arrive, such as a pair of heaps — one holding the smaller half and one holding the larger half of the data seen so far, from Week 8. This reflects the general engineering principle established by this lecture's sort-then-index-versus-`quickselect` comparison: the right choice of algorithm always depends on the actual query pattern, not on any one algorithm being universally superior. One-shot, single queries against otherwise-static data favor `quickselect`'s expected linear time; repeated queries against data that changes slowly over time favor investing in a structure that maintains order incrementally instead.

Exercise

Implement a `quickselect_median(arr)` function that returns the median of a list containing an odd number of elements, built directly on top of the `quickselect` function introduced in this lecture. Verify the implementation against `sorted(arr)[len(arr)//2]` on at least five different test lists of varying sizes and contents, confirming the two approaches always agree.

Quiz — expected time

Given that both quicksort and quickselect partition the array around a randomly chosen pivot in essentially the same way, explain precisely why quicksort's expected running time is $O(n \log n)$ while quickselect's expected running time is $O(n)$.

Quiz — answer

Quicksort recurses on both sides of every partition, so the same total amount of partitioning work is effectively repeated across $\log n$ levels of recursion, yielding $O(n \log n)$ overall. Quickselect recurses on only the one side known to contain the target position, discarding the other side entirely, so the total work across all recursive calls forms a convergent geometric series that sums to $O(n)$. The key difference is not the partitioning step itself, which is essentially identical in both algorithms, but what happens immediately after partitioning: recursing into both remaining pieces, versus discarding one and recursing into only the other.

Exercise — answer sketch

Since the `quickselect` function developed earlier in this lecture already handles an arbitrary target position k , `quickselect_median` reduces to a one-line wrapper that calls it with $k = n // 2$. Testing this against five different lists — an ordinary five-element list, a list containing a duplicate value, a single-element list, a five-element list of distinct values, and a list where every element is identical — confirms the result agrees exactly with `sorted(arr)[len(arr)//2]` in every case, including the edge cases of a single element and entirely repeated values.

Looking ahead: Week 15

Week 15 is Project Presentations and Course Review, a different format from the uniform four-lecture structure used throughout Weeks 2 through 14. It is intended to bring together concepts spanning the entire course: fundamental data structures from Weeks 2 through 4 and 7, sorting algorithms from Weeks 5 and 6, trees and hashing from Weeks 6 through 8, dynamic programming and greedy algorithms from Weeks 9 and 10, graph algorithms from Weeks 11 through 13, and parallel and randomized algorithms from this week. Course project presentations are expected to demonstrate a real application that combines at least two of these algorithmic ideas, rather than treating any single week's material in isolation.

Summary

Quickselect adapts quicksort's own partition step to find the k -th smallest element of an unsorted list in expected $O(n)$ time, achieving this by discarding, at each step, the side of the partition that cannot contain the target position. The median-of-medians algorithm trades a larger practical constant factor for a guaranteed worst-case $O(n)$ bound, directly illustrating the guaranteed-versus-expected performance trade-off that echoes Lecture 2's Las Vegas discussion. Taken as a whole, Week 14 has shown that parallelism, covered in Lectures 1 and 3, and randomization, covered in Lectures 2 and 4, are two distinct but complementary techniques for making algorithms faster than their naive sequential, fully deterministic counterparts. This closes the algorithmic core of Programming and Data Structures; Week 15 turns to project presentations and a review of the course as a whole.